# ECO5050 AI Applications and Techniques in Economics

## Lecture 7 Regression Trees, Random Forests, and Hyperparameter selection

Yaein Baek

Sogang University

Fall 2025

# Outline

# Nonparametric Regression

- Nonparametric methods allows us to approximate the features of the joint distribution of $Z$with minimal information on the underlying model
  - For MSE loss, we approximate the conditional mean of $Y$ given $Z$ and for lin-lin loss it is a conditional quantile
- Nonparameteric approaches aim to find methods that work for a wide class of functions, hence there is a greater chance that the estimated forecasting model is close to the optimal model
- Brief introduction of two approaches
  - Kernel estimation
  - Sieve estimation

# Nonparametric Regression

## Kernel estimation: Binned means estimator

- The conditional expectation function $m(x)$ can take any nonlinear shape, and is therefore nonparametric

$$m(x) \equiv E[Y|X = x]$$

- Fix the point $x$ and consider estimation of $m(x)$. If $X$ is continuous, the probability is 0 that $X$ exactly equals $x$ so there is no subsample of observations with $X_t = x$.
- If $m(x)$ is continuous, it is possible to get a good approximation by taking the average of the observations for which $|X_t - x| \leq h$ for some bandwidth $h > 0$

$$\hat{m}(x) = \frac{\sum_{t=1}^{T} \mathbf{1}\{|X_t - x| \leq h\} Y_t}{\sum_{t=1}^{T} \mathbf{1}\{|X_t - x| \leq h\}}$$

- The binned means estimator $\hat{m}(x)$ is a step function in $x$, there is discontinuity

# Nonparametric Regression

## Kernel estimation

- Weights can be continuous functions so that $\hat{m}(x)$ is also continuous in $x$
- Appropriate weight functions are called kernel functions
- The Nadaraya-Watson (NW) or kernel regression estimator

$$\hat{m}_{NW}(x) = \frac{\sum_{t=1}^{T} K\left(\frac{X_t - x}{h}\right) Y_t}{\sum_{t=1}^{T} K\left(\frac{X_t - x}{h}\right)}$$

- The NW estimator is also called a local constant estimator because it locally (about $x$) approximates $m(x)$ as a constant function.

$$\hat{m}_{NW}(x) = \arg\min_{m} \sum_{t=1}^{T} K\left(\frac{X_t - x}{h}\right) (Y_t - m)^2$$

This is a weighted regression of $Y$ on an intercept only.

# Nonparametric Regression

## Kernel estimation

**Table 19.1**
Common normalized second-order kernels

| Kernel | Formula | $R_K$ |
|---|---|---|
| Rectangular | $K(u) = \begin{cases} \frac{1}{2\sqrt{3}} & \text{if } \lvert u\rvert < \sqrt{3} \\ 0 & \text{otherwise} \end{cases}$ | $\frac{1}{2\sqrt{3}}$ |
| Gaussian | $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$ | $\frac{1}{2\sqrt{\pi}}$ |
| Epanechnikov | $K(u) = \begin{cases} \frac{3}{4\sqrt{5}}\left(1 - \frac{u^2}{5}\right) & \text{if } \lvert u\rvert < \sqrt{5} \\ 0 & \text{otherwise} \end{cases}$ | $\frac{3\sqrt{5}}{25}$ |
| Triangular | $K(u) = \begin{cases} \frac{1}{\sqrt{6}}\left(1 - \frac{\lvert u\rvert}{\sqrt{6}}\right) & \text{if } \lvert u\rvert < \sqrt{6} \\ 0 & \text{otherwise} \end{cases}$ | $\frac{\sqrt{6}}{9}$ |
| Biweight | $K(u) = \begin{cases} \frac{15}{16\sqrt{7}}\left(1 - \frac{u^2}{7}\right)^2 & \text{if } \lvert u\rvert < \sqrt{7} \\ 0 & \text{otherwise} \end{cases}$ | $\frac{5\sqrt{7}}{49}$ |

Hansen (2022)

# Nonparametric Regression

## Kernel estimation of forecasting models

- Local regression methods are essentially weighted least square estimators that assign large weights to previous data for which the predictor variables are "close" to the values observed at the point where the current forecast is made.
- Example: The conditional mean estimate, hence the forecast at time $T$ is

$$f_{T+1|T}(z_T) = \frac{\sum_{t=0}^{T-1} y_{t+1} K(z_t - z_T)}{\sum_{t=0}^{T-1} K(z_t - z_T)}$$

$$K(z_t - z_T) = |H|^{-1/2} \exp\left(-\frac{1}{2}(z_t - z_T)' H^{-1}(z_t - z_T)\right)$$

$K(z_t - z_T)$ is a multivariate Gaussian kernel and $H$ is a bandwidth matrix chosen by the forecaster.

# Nonparametric Regression

## Sieve estimation: Series regression

- Sieve estimation seeks to approximate an unknown function by combinations of functions of the data.
- Includes series estimators and spline estimators as special cases.
- A series regression model is a sequence $K = 1, 2, \ldots,$ of approximating models $m_K(x)$ with $K$ parameters.
- Consider series approximation to $m(x) = E[Y|X = x]$

$$m_K(x) = X_K(x)'\beta_K$$

  where $X_K$ is a vector of regressors obtained by making transformations of $X$ and $\beta_K$ is a coefficient vector
- The estimator for $m(x)$ is $\hat{m}_K(x) = X_K(x)'\hat{\beta}_K$, where the $\hat{\beta}_K$ is typically a LS estimator.

# Nonparametric Regression

## Sieve estimation: Series regression

- In general, a linear series regression model takes the form

$$m_K(x) = \sum_{j=1}^{K} \beta_j \tau_j(x)$$

  where the functions $\tau_j(x)$ are the basis transformations

- Polynomial regression model for $m(x)$ has power basis $\tau_j(x) = x^{j-1}$

$$m_K(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p$$

  Then $X_K(x) = (1, x, x^2, \cdots, x^p)'$

# Nonparametric Regression

## Sieve estimation: Splines

- A spline is a piecewise polynomial, it provide a method for effectively partitioning the space $\{y, x\}$ so that different models are estimated on separate partitions but remain continuous over the full range of $x$
  - The joint points between the partitions are called knots
  - Usually assume the spline function has continuous derivatives up to the order of the spline
- A quadratic spline with one knot

$$m_K(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 (x - \tau)^2 \mathbf{1}\{x \geq \tau\}$$

- In general, a $p$th-order spline with $N$ knots $\tau_1 < \tau_2 < \cdots < \tau_N$ is

$$m_K(x) = \sum_{j=0}^{p} \beta_j x^j + \sum_{k=1}^{N} \beta_{p+k} (x - \tau_k)^p \mathbf{1}\{x \geq \tau_k\}$$

which has $K = N + p + 1$ coefficients

# Nonparametric Regression

## Sieve estimation: Splines

- In practice, a spline will depend critically on the choice of the knots $\tau_k$
  - If X is bounded with an approximately uniform distribution, space the knots evenly so that all segments have the same length.
  - If X is not uniformly distributed, set the knots at the quantiles $j/(N+1)$ so that the probability mass is equalized across segments.
  - An alternative is to set the knots at the points where $m(x)$ has the greatest change in curvature
- In all cases, the set of knots $\tau_j$ change with $K$; a spline is a special case of an approximation of the form

$$m_K(x) = \beta_1 \tau_{1K}(x) + \beta_2 \tau_{2K}(x) + \cdots + \beta_K \tau_{KK}(x)$$

where the basis transformation $\tau_{jK}(x)$ depends on both $j$ and $K$

# Regression Trees

- Breiman et al. (1984) introduced "CART" (Classification and Regression Trees).
- A regression tree is a nonparametric regression using a large number of step functions
  - Idea: With a sufficiently large number of split points, a step function can approximate any function
  - Threshold regression with intercept only

$$Y = \mu_1 \mathbf{1}\{X_k \leq \gamma\} + \mu_2 \mathbf{1}\{X_k > \gamma\} + u$$

  - Zero-th order spline with free knots
- Useful when there are a combination of continuous and discrete regressors so that traditional kernel and series methods are difficult to implement.
- Estimate $m(\mathbf{x}) = E[Y|\mathbf{X} = \mathbf{x}]$, where the elements of $\mathbf{X}$ can be continuous, discrete, or ordinal.

# Regression Trees

- Split the sample $\{(X_{1t}, \ldots, X_{Nt}, Y_{t+1})\}_{t=0}^{T-h}$ into subsamples and estimate the regression function within the subsamples as the average outcome
- The splits are sequential and based on a single regressor $X_{it}$ exceeding a threshold $c$

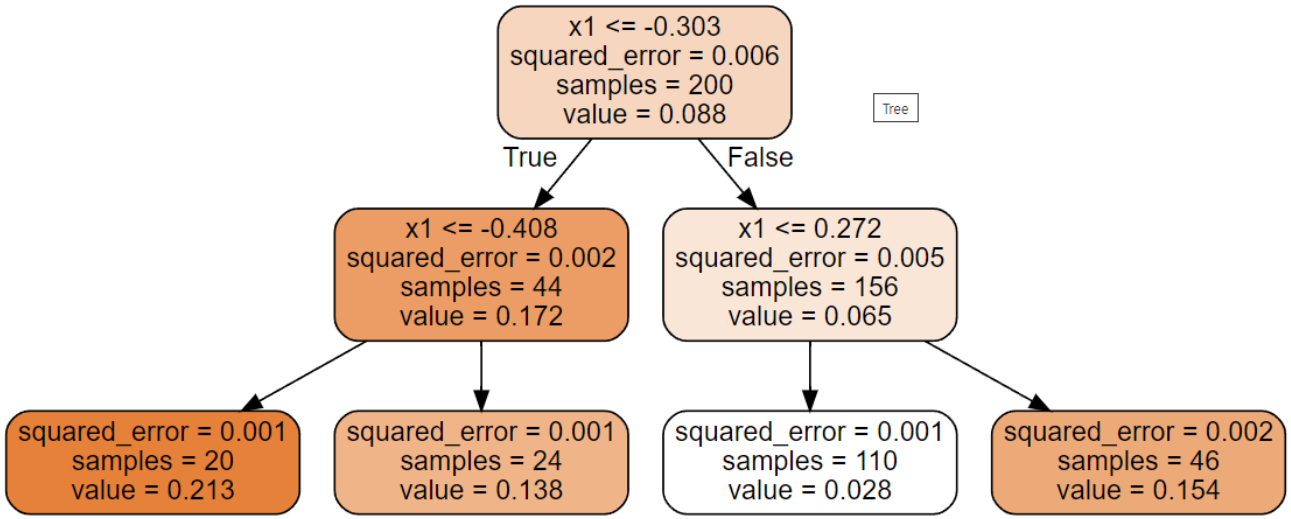
$$Q(k, c) = \sum_{t:X_{it} \leq c} (Y_{t+1} - \bar{Y}_{k,c,l})^2 + \sum_{t:X_{it} > c} (Y_{t+1} - \bar{Y}_{k,c,r})^2$$

$$\bar{Y}_{k,c,l} = \sum_{t:X_{it} \leq c} Y_{t+1} \Big/ \sum_{t:X_{it} \leq c} 1$$

$$\bar{Y}_{k,c,r} = \sum_{t:X_{it} > c} Y_{t+1} \Big/ \sum_{t:X_{it} > c} 1$$

- Split the sample using the regressor $i$ and threshold $c$ that minimize the average squared error $Q(k, c)$ over all $i = 1, \ldots, N$ and $c \in (-\infty, \infty)$
- Repeat this, optimizing over the subsamples

# Regression Trees




Géron (2023) Figure 6-4

# Regression Trees

- We can interpret a regression tree as an alternative to kernel regression

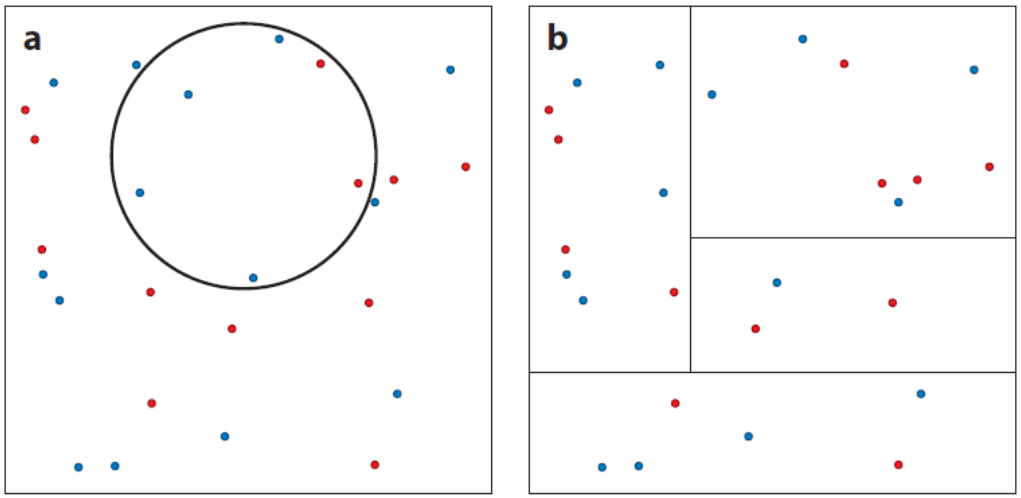



Figure 1

(*a*) Euclidean neighborhood for *k*-nearest neighbor (KNN) matching. (*b*) Tree-based neighborhood.

Athey and Imbens (2019)

- Kernel regression creates a neighborhood around a target observation $x$ based on the Euclidean distance to each point
- Regression tree based neighborhoods will be rectangles, the leaf defines the set of nearest neighbors for $x$

# Regression Trees

- The literature on regression trees uses the following terminologies
  - A subsample is a **branch**
  - Terminal branches are **nodes** or **leaves**
  - Increasing the number of branches is **growing** a tree
  - Decreasing the number of branches is **pruning** a tree
- At each split the mean squared error is further reduced (or stays the same) so we need some regularization to avoid overfitting
  - Example: Add a penalty term to the sum of squared residuals that is a linear function of the number of leaves

# Regression Trees

## Regularization hyperparameters in Python scikit-learn

`DecisionTreeRegressor()` ▸ Document

- `max_depth`: Maximum depth of the tree
- `max_features`: Maximum number of features (predictors) that are evaluated for splitting each node
- `max_leaf_nodes`: Maximum number of leaf nodes
- `min_samples_split`: Minimum number of samples a node must have before it can be split
- `min_samples_leaf`: Minimum number of samples required to be a leaf node
- `min_weight_fraction_leaf`: Same as `min_samples_leaf` but expressed as a fraction of the total number of samples

# Regression Trees

## Regularization hyperparameters in Python scikit-learn

`DecisionTreeRegressor()` ▸ Document

- `ccp_alpha`: Complexity parameter $\alpha \geq 0$ used for Minimal Cost-Complexity Pruning. Similar to the Mallows-type information criterion:

$$R_\alpha(T) = R(T) + \alpha|\tilde{T}|$$

where $R(T)$ is the total misclassification rate of the terminal nodes, $|\tilde{T}|$ is the number of terminal nodes

# Regression Trees

## Python scikit-learn's API

`DecisionTreeRegressor()` ▸ Document

- `random_state`: Controls the randomness of the estimator. When `max_features` $<$ `n_features`, the algorithm will select `max_features` at random at each split before finding the best split among them.
  - default=None
  - Fix to an integer for reproducibility of the results returned by the function

# Regression Trees for Classification

## Python scikit-learn's API

`DecisionTreeClassifier()` ▸ Document

- Use if the label is binary or multiclass ($Y = 1, \ldots, n$)
- The criterion are impurity measures that measure the quality of the split
  - The default `criterion="gini"` measures the Gini impurity

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

$G_i$ is the Gini impurity of the $i$th node, $p_{i,k}$ is the ratio of class $k$ observations among the training sample in the $i$th node

  - `criterion="entropy"` measures the entropy

$$H_i = - \sum_{k=1, p_{i,k} \neq 0}^{n} p_{i,k} \log_2(p_{i,k})$$

# Regression Trees

- An advantage of a RT is that it is easy to interpret and provides a highly flexible nonparametric approximation.
  - However it is important not to go too far in interpreting the structure of the tree, including the selection of variables used for splits.
- A disadvantage is that the estimate $\hat{m}(x)$ is a discrete function, it may be crude approximation when $m(x)$ is continuous and smooth.
  - To obtain a good approximation RT may require a large number of leaves, which can result in a nonparsimonious model with high estimation variance.
- RTs have high variance; small changes to the hyperparameters or to the data may produce very different models.
- Generalization of tree-based methods can be used: Random Forests

# Random Forests

- Random Forests (RF) is designed to reduce estimation variance of simple regression trees; it induce smoothness by averaging over a large number of trees.
- Apply bagging to regression trees and introduce extra randomness to decorrelate the bootstrap trees (two sources of randomness in trees)
  - Each tree is based on a bootstrap sample, or a subsample
  - The splits at each stage are not optimized over all possible regressors, but over a random subset of regressors that change in every split
- RF requires little tuning and has good out-of-sample performance compared to more complex methods such as deep learning neural networks
- RT and RF are particularly effective in sparse settings, where there is a large number of predictor variables that are not related to the outcome

# Random Forests

The basic RF algorithm (Hastie, Tibshirani, and Friedman, 2009)

1. For $b = 1, \ldots, B$
   a. Draw a bootstrap sample from the training data.
   b. Grow a regression tree on the bootstrap sample by recursively repeating the following steps until the minimum leaf size $n_{\text{min}}$ is reached.
      i. Select $p$ variables at random from the $N$ regressors ($p < N$)
      ii. Among these $p$ variables, pick the one that produces the best regression split
      iii. Split the bootstrap sample accordingly
   c. Set $\hat{m}_b(x)$ as the sample mean of $Y$ on each leaf of the bootstrap tree.
2. $\hat{m}_{RF}(x) = B^{-1} \sum_{b=1}^{B} \hat{m}_b(x)$

# Random Forests

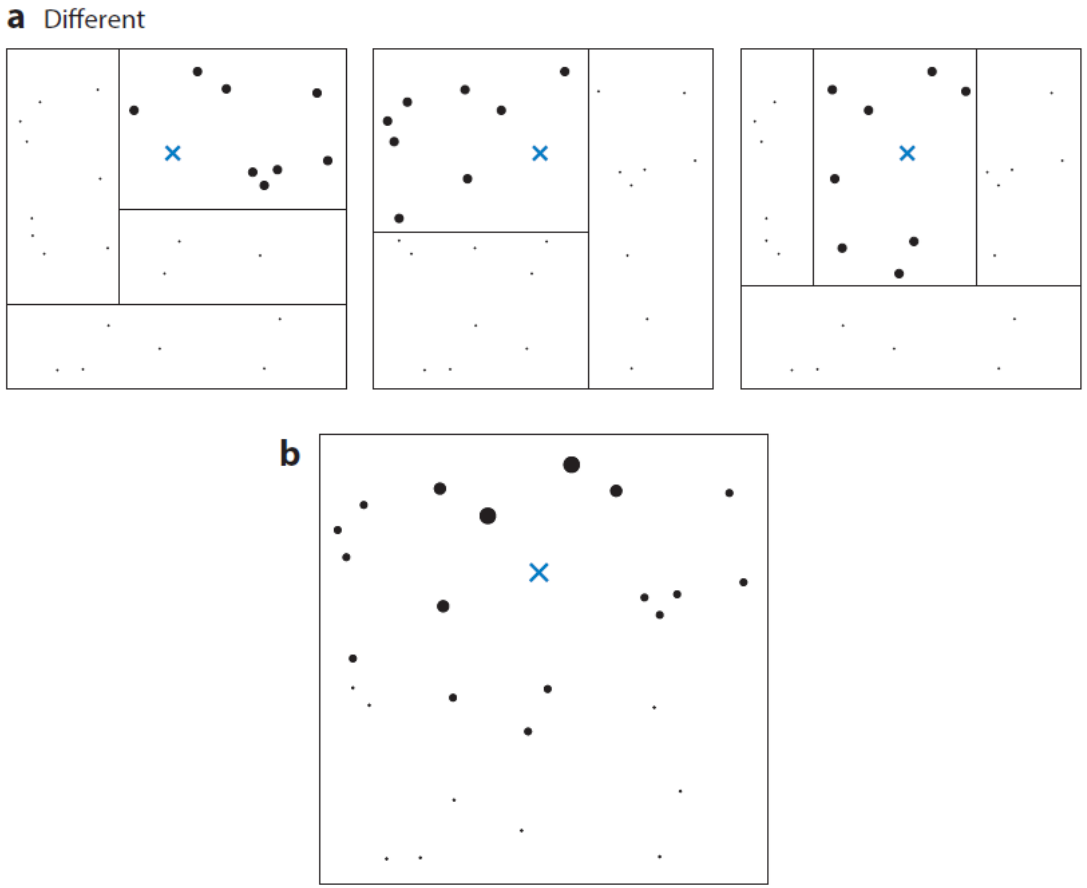



Figure 2

(*a*) Different trees in a random forest generating weights for test point $X$. (*b*) The kernel based on the share of trees in the same leaf as test point $X$.

Athey and Imbens (2019)

# Random Forests

- RF averages over many trees, will include nearby points more often than distant points.
- This is analogous to kernel weighting functions; a kernel regression makes a prediction at a point $x$ by averaging nearby points but weighting closer points more heavily.
- We can derive the weighting function for a given test point $x$ by counting the share of trees where a particular observation is in the same leaf as a test point

$$\hat{m}_{RF}(x) = \sum_{t=1}^{T} \alpha_t(x) Y_t, \quad \sum_{t=1}^{T} \alpha_t(x) = 1, \quad \alpha_t(x) \geq 0$$

where the weights $\alpha_t(x)$ encode the weight given by the forest to the $t$th training sample when predicting $x$

# Random Forests

- Wager and Athey (2018) establishes consistency and asymptotic normality of RF estimators, based on RF generated by subsampling than bootstrap
  - A subsampling estimator is based on sampling without replacement rather than with replacement, as done in the bootstrap
- The sampling distribution of regression trees is difficult to derive, in part because of the strong correlation between the placement of the sample splits and the estimated means.
- Wager and Athey (2018) proposes honest trees: Split the sample into two halves A and B, use the A sample to place the splits and the B sample to do within-leaf estimation
- Assume that the RF is created by subsampling, estimated by honest trees, and that the minimal split fraction satisfies $0 < \alpha \leq 0.2$, then pointwise in $x$,

$$\frac{\hat{m}_{RF}(x) - m(x)}{\sqrt{V_T(x)}} \xrightarrow{d} N(0,1)$$

for some variance sequence $V_T(x) \to 0$

# Random Forests

- The disadvantage of RF is that they are not very efficient at capturing linear or quadratic effects or at exploiting smoothness of the underlying data-generating process (Athey and Imbens, 2019)
- In addition, the estimators for points near the boundary of the covariate space are likely to have bias because the leaves of the trees cannot be centered on points near the boundary
- Friedberg et al. (2020) proposes local linear forests, which are constructed by running a regression weighted by the weighting function derived from a forest.

# Random Forests

## Hyperparameters in Python Scikit-Learn

`RandomForestRegressor()` ▸ Document

- Same hyperparameters as `DecisionTreeRegressor()`
  - `max_depth`, `max_leaf_nodes`, `min_samples_leaf`, `min_samples_split`, `min_weight_fraction_leaf`, `ccp_alpha`
- `n_estimators`: The number of trees in the forest
- `max_features`: The default of 1.0 is equivalent to bagged regression trees; set to values smaller than 1.0
  - If it is set to an integer $> 1.0$, it is the number of features considered at each split

# Random Forests

## Python Scikit-Learn's API

`RandomForestRegressor()` ▸ Document

- `criterion`: default= "squared_error"
- `bootstrap`: Whether bootstrap samples are used for building trees
  - default=True
- `max_samples`: The size of each bootstrap sample to draw from $X$ to train each tree if `bootstrap=True`
  - default=None; the bootstrap sample size is equal to the size of the original sample
- `random_state`: Controls both the randomness of the bootstrapping of the samples used when building trees (if `bootstrap=True`) and the sampling of the features to consider when looking for the best split at each node (if `max_features` $<$ `n_features`)
  - default=None

# Random Forests

## Python Scikit-Learn's API

- `ExtraTreesRegressor()`: Random Forest using Extra-Trees ▸ Document
  - `bootstrap` defaults to `False`
- Extremely randomized trees (Extra-Trees): Make trees even more random by using random thresholds for each feature than searching for the best possible thresholds
  - Instead of the default `splitter="best"`, generates trees by `DecisionTreeRegressor(splitter="random")`
  - Trades more bias for a lower variance

# Random Forests

## Python Scikit-Learn's API: Feature importance

- One possible measure of the importance of a feature is the total reduction of the criterion brought by that feature on average, across all trees in the forest
- `feature_importances_` computes this score for each feature after training and normalizes it so that the sum of all importances is equal to 1
- Importance of feature $X$

$$\text{FI}(X) = \frac{1}{n} \sum_{j=1}^{n} \sum_{k \in \text{ nodes where } X \text{ is used}} \frac{\Delta MSE_{j,k}}{\text{Total MSE reduction in tree } j}$$

  $n$ is the total number of trees, $\Delta MSE_{j,k}$ is the reduction in MSE at node $k$ in tree $j$ where feature $X$ is used to split

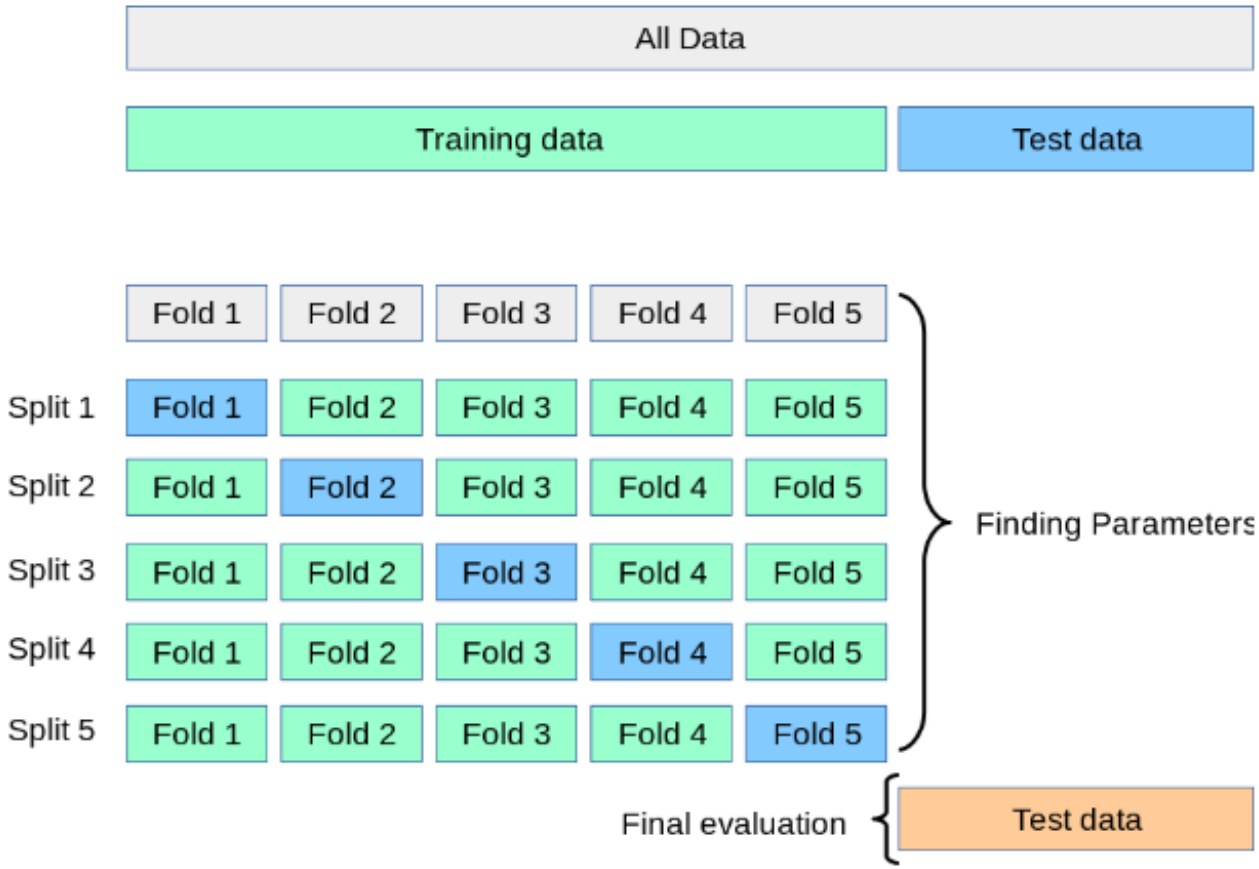
- It can give more biased importance to features with more unique values (e.g. continuous features over categorical ones) and may not reflect the true contribution of the feature to predictive accuracy
- Example: ▸ Link

# Hyperparameter Selection

- Most Machine Learning Algorithms need to specify hyperparameters (fine-tuning)
  - RT: `max_depth = 3, 4` or larger?
  - RF: How many trees to set in `n_estimators`?
- This is equivalent to a model selection process, we want to select the optimal values of hyperparameters $\rightarrow$ use cross-validation?
  - We also want to compute the out-of-sample MSE to evaluate predictive performance
  - Using the same data set for model selection and performance evaluation can lead to optimistic results
- Use the **nested cross-validation** for model selection (fine-tuning) and performance evaluation of Machine Learning forecasts.
  - Typically use nested $K$-fold cross-validation

# Hyperparameter Selection

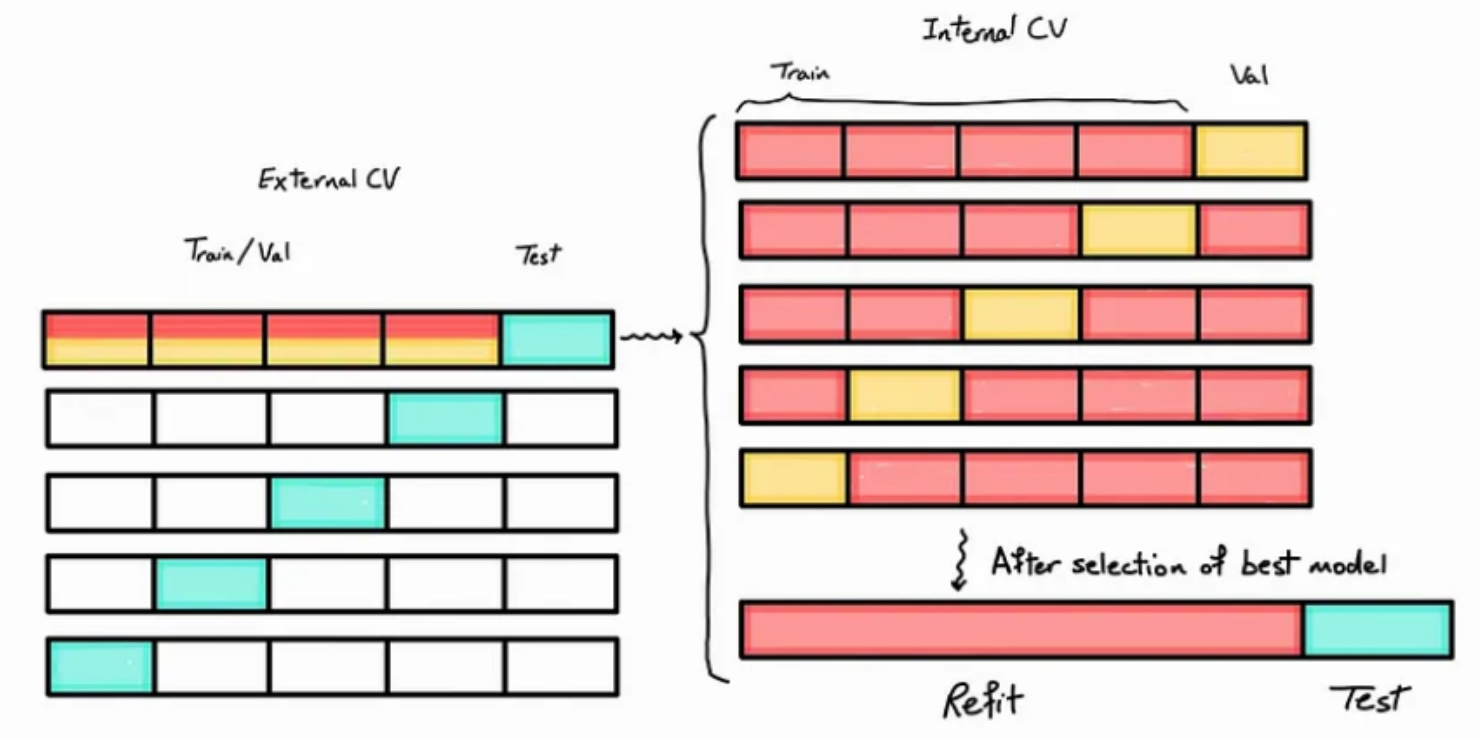
## Nested cross-validation




Source: ▸ Link

# Hyperparameter Selection

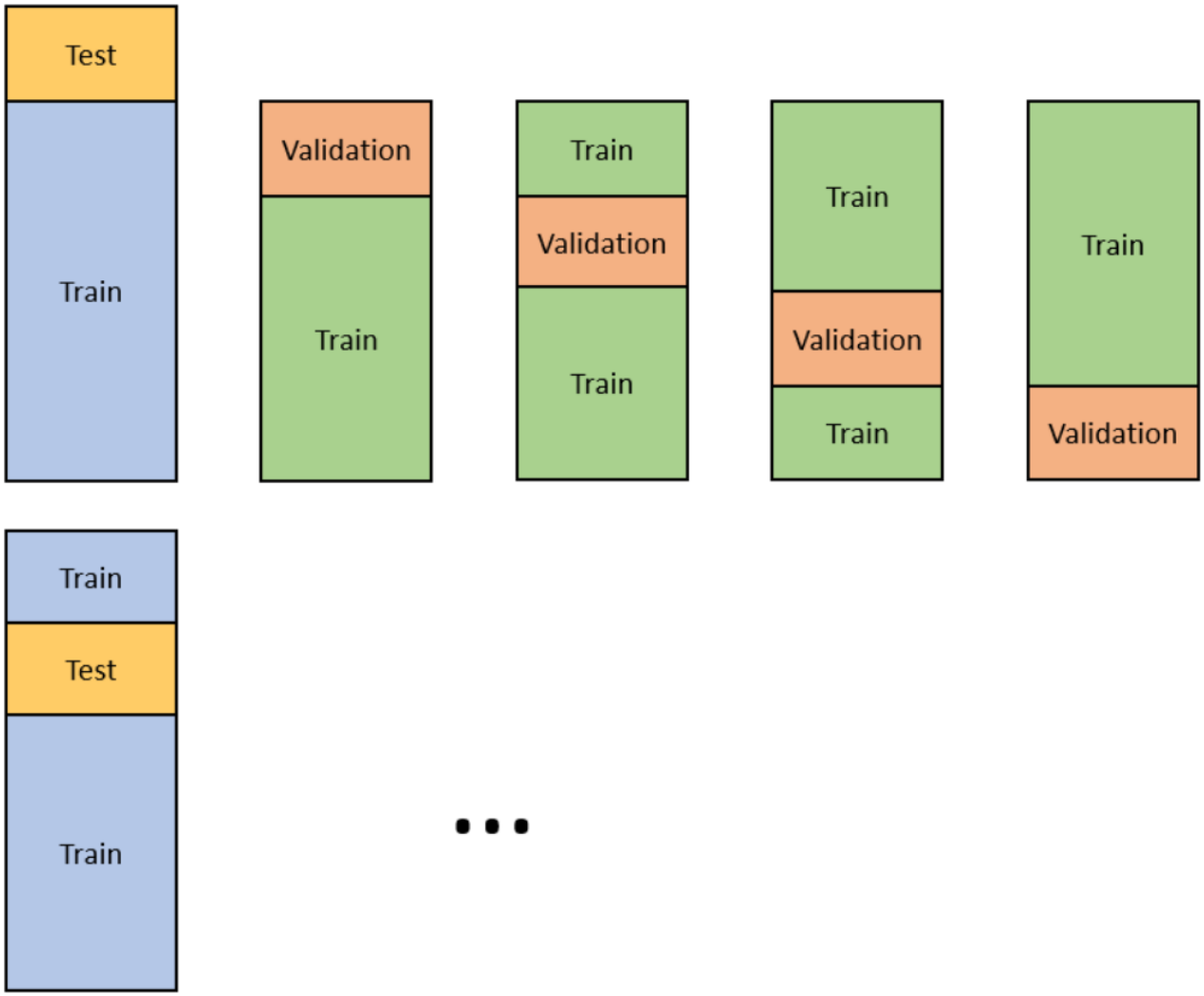
## Nested $K$-fold cross-validation




Source: ▸ Link

# Hyperparameter Selection

## Nested $K$-fold cross-validation




Source: ▸ Link

# Hyperparameter Selection

## Nested $K$-fold cross-validation

1. Set a grid of hyperparameters $\alpha \in \{\alpha_1, \alpha_2, \ldots, \alpha_m\}$.
2. The outer $K$-fold CV splits the data into a training set ($K-1$ folds) and a test set (1 fold).
3. The inner CV splits the (outer) training set into $K$ folds, the (inner) training and validation sets.
   - For each $\alpha$, the model is fitted into the inner training set and evaluated on the validation set
   - If the inner CV is 5-fold, the model is fitted and evaluated on 5 different training and validation sets
4. Pick the optimal hyperparameter $\alpha^*$ that minimizes the evaluation criterion (e.g. out-of-sample MSE).
5. Fit the model with $\alpha^*$ on the outer training set, which is the inner training and validation sets combined together.
6. Compute the evaluation criterion of the model on the test set.
7. Repeat 3-6 for the splits in the outer CV in 2.

# Hyperparameter Selection

## Scikit-Learn `GridSearchCV()`

- Find the optimal combination of hyperparameter values via exhaustive grid search using cross-validation
- `GridSearchCV(estimator, param_grid, cv=None)` ▸ Document
  - Example: Random Forest

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

rf_model = RandomForestRegressor(random_state=1)

rf_param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [3, 5, 7],
    'max_features': [0.3, 0.5, 0.8]
}

rf_search = GridSearchCV(estimator=rf_model,
                         param_grid = rf_param_grid,
                         scoring='neg_mean_squared_error',
                         cv=5,
                         n_jobs=-1)

rf_opt_model = rf_search.fit(X_train, y_train)
print(rf_opt_model.best_params_)
```

```
{'max_depth': 3, 'max_features': 0.3, 'n_estimators': 300}
```

# Hyperparameter Selection

## Scikit-Learn `GridSearchCV()`

- Example: Lasso regression

```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Lasso

lasso_pipeline = make_pipeline(StandardScaler(),
                               Lasso())

lasso_param_grid = {
    'lasso__alpha': [0.01, 0.05, 0.1, 0.5, 1.0]
}

lasso_search = GridSearchCV(estimator=lasso_pipeline,
                            param_grid=lasso_param_grid,
                             scoring='neg_mean_squared_error',
                             cv=5,
                             n_jobs=-1)

lasso_opt_model = lasso_search.fit(X_train, y_train)
best_lasso_model = lasso_search.best_estimator_
lasso_forecast = best_lasso_model.predict(X_test)

print("Best parameters found:", lasso_search.best_params_)
print(f'Negative out-of-sample MSE: {lasso_opt_model.score(X_test, y_test): .4f}')
```

```
Best parameters found: {'lasso__alpha': 0.05}
Negative out-of-sample MSE: -0.0071
```

# Hyperparameter Selection

## Scikit-Learn `GridSearchCV()`

`GridSearchCV(estimator, param_grid, cv=None)` ▸ Document

- `cv` = integer, cross-validation generator (CV splitter) or an iterable
  - `None`: use the default 5-fold cross-validation
  - integer $k$: specify the number of $k$ folds in `KFold`
  - CV splitter: a non-estimator family of classes used to split a dataset into a sequence of train and test portions by providing `split` and `get_n_splits` methods, e.g. `PreDefinedSplit()`

# Hyperparameter Selection

## Example: `PredefinedSplit()`

- Example: `cv` = cross-validation generator using `PredefinedSplit()`

```
T, _ = X_train.shape

n_train = np.floor(T*2/3).astype(int)
n_valid = T-n_train

train_indices = np.full((n_train,), -1, dtype=int)
valid_indices = np.full((n_valid,), 0, dtype=int)
valid_fold = np.append(train_indices, valid_indices)
ps = PredefinedSplit(valid_fold)

rf_model = RandomForestRegressor(random_state=1)

rf_param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [3, 5, 7],
    'max_features': [0.3, 0.5, 0.8]
}

rf_search = GridSearchCV(estimator=rf_model,
                         param_grid = rf_param_grid,
                         scoring='neg_mean_squared_error',
                         cv=ps,
                         n_jobs=-1)

rf_opt_model = rf_search.fit(X_train, y_train)
best_model_rf = rf_opt_model.best_estimator_
rf_forecast = best_model_rf.predict(X_test)
```

- Split the outer training set: the first 2/3 samples in the inner training set and latter 1/3 samples in the validation set
- The indices which have the value -1 will be kept in the training set, zero or positive values will be kept in the validation set

# Hyperparameter Selection

## Example: CV splitter

- Example: cv = cross-validation generator

```
class BlockvKFold:
    def __init__(self, n_splits=5, v=3):
        self.n_splits = n_splits
        self.v = v

    def split(self, X, y=None, groups=None):
        kf = KFold(n_splits=self.n_splits, shuffle=False)
        indices = np.arange(len(X))
        for train_index, test_index in kf.split(X):
            # Exclude the prior and post v observations from train_index
            start_val, end_val = test_index[0], test_index[-1]
            exclude_indices = np.arange(max(start_val - self.v, 0),
                                        min(end_val + self.v + 1, len(X)))
            train_index = np.setdiff1d(indices, exclude_indices)
            yield train_index, test_index

    def get_n_splits(self, X, y=None, groups=None):
        return self.n_splits


block_cv = BlockvKFold(n_splits=5, v=3)
```

- Split the outer training set into $K$ folds: the $v$ observations on each side of the validation set are discarded
- The $v$ observations on each side are omitted in order to account for dependence between the training and validation sets.
- Similar to the $hv$-block CV

# Hyperparameter Selection

```python
from sklearn.metrics import mean_squared_error

# Set up the outer cross-validation
outer_cv = KFold(n_splits=5, shuffle=False)

# List to store MSE from the outer CV
outer_mse = []


for train_idx, test_idx in outer_cv.split(X, y):

    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    # Define the inner cross-validation and GridSearchCV
    inner_cv = block_cv
    grid_search = GridSearchCV(estimator=rf_model,
                               param_grid=rf_param_grid,
                               scoring='neg_mean_squared_error',
                               cv=inner_cv,
                               n_jobs=-1)

    grid_search.fit(X_train, y_train)

    best_rf_model = grid_search.best_estimator_
    y_pred = best_rf_model.predict(X_test)
    test_mse = mean_squared_error(y_test, y_pred)
    outer_mse.append(test_mse)


    # Print the best parameters and score from the inner loop for each fold
    print(f"Best parameters in inner fold: {grid_search.best_params_}")
    print(f"MSE on outer fold: {test_mse}")

# Calculate and print the average MSE across all outer folds
oos_mse = np.mean(outer_mse)
print(f"\nAverage MSE across all outer folds: {oos_mse}")
```

- Example: Nested CV
- Also check the example in

  ▸ Document

Athey, S., & Imbens, G. W. (2019).Machine learning methods that economists should know about. *Annual Review of Economics*, *11*, 685–725.

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and regression trees* (1st ed.). Chapman and Hall.

Friedberg, R., Tibshirani, J., Athey, S., & Wager, S. (2020).Local linear forests. *Journal of Computational and Graphical Statistics*, *30*(2), 503–517.

Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras & tensorflow* (2nd ed.). O'Reilly Media, Inc.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

Wager, S., & Athey, S. (2018).Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, *113*(523), 1228–1242.